

Estratégia para um Passo de Advecção Semi-Lagrangeana no `femns`

Projeto `femns` – documento de discussão (não implementado)

Julho de 2026

Resumo

Este documento é um resumo de estratégia, não uma especificação final: descreve como um passo de advecção semi-Lagrangeana poderia substituir o tratamento explícito atual do termo convectivo em `femns`, mapeando cada etapa do método para módulos concretos do projeto. O objetivo é servir de base de discussão antes de qualquer mudança na numérica do solver.

Sumário

1 Contexto: como a advecção é tratada hoje	1
2 Ideia central do método	1
2.1 O sistema em forma de blocos: antes e depois	2
3 Estratégia de implementação proposta	2
3.1 Passo 1 – Backtrace: encontrar o ponto de partida	3
3.2 Passo 2 – Localizar o elemento que contém o ponto de partida	3
3.3 Passo 3 – Interpolar a velocidade no pé da característica	4
3.4 Passo 4 – Trocar o termo convectivo em <code>solver.py</code>	5
3.5 Pontos de partida fora do domínio	6
3.6 Vetorização em GPU / <code>torch</code>	6
3.7 Testes	6
3.8 Estratégia de transição e validação	6
4 Impacto esperado e trade-offs	6
5 Referências	7

1 Contexto: como a advecção é tratada hoje

Em `solver.time_step` (`src/femns/solver.py`), o termo convectivo linearizado é montado explicitamente a cada passo e jogado para o lado direito do sistema:

```
vg = torch.mm(vx_diag, Gvx) + torch.mm(vy_diag, Gvy)
b_sup = (1/dt) * torch.mm(M, vx.unsqueeze(1)) - torch.mm(vg, vx.unsqueeze(1))
b_mid = (1/dt) * torch.mm(M, vy.unsqueeze(1)) - torch.mm(vg, vy.unsqueeze(1))
```

Ou seja: a matriz do sistema A (`build_system_matrix`) só contém os blocos de massa/rigidez ($M/dt + K/Re$) e o acoplamento de pressão – ela *não* contém advecção. O termo advectivo é avaliado com o campo de velocidade do passo anterior e movido para b . Isso equivale a tratar

a advecção de forma **explícita** (tipo Euler explícito) enquanto difusão e pressão continuam implícitas.

O efeito prático: o passo de tempo Δt fica limitado por uma condição do tipo CFL ($\Delta t \lesssim h/|v|$) para a parte advectiva, mesmo a parte difusiva/pressão sendo incondicionalmente estável. Isso é a motivação natural para semi-Lagrangeano: o método trata a advecção de forma que não impõe essa restrição de CFL – permitindo Δt maior – ao custo de introduzir difusão numérica pela interpolação.

2 Ideia central do método

A equação de quantidade de movimento tem a forma

$$\frac{D\mathbf{v}}{Dt} = \frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla)\mathbf{v} = -\nabla p + \frac{1}{Re} \nabla^2 \mathbf{v}.$$

O método semi-Lagrangeano discretiza a **derivada material** diretamente, em vez de discretizar separadamente o termo temporal e o convectivo:

$$\left. \frac{D\mathbf{v}}{Dt} \right|_{\mathbf{x}_n}^{t^{n+1}} \approx \frac{\mathbf{v}^{n+1}(\mathbf{x}_n) - \mathbf{v}^n(\mathbf{x}_d)}{dt},$$

onde $\mathbf{x}_d$ é o **ponto de partida** (*departure point*): a posição de onde a partícula de fluido que chega em $\mathbf{x}_n$ no tempo t^{n+1} partiu no tempo t^n , seguindo a característica

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}(\mathbf{x}, t), \quad \mathbf{x}(t^{n+1}) = \mathbf{x}_n.$$

Como $\mathbf{v}^n(\mathbf{x}_d)$ não cai sobre um nó da malha, ele precisa ser **interpolado** a partir dos valores nodais nos vértices do elemento que contém $\mathbf{x}_d$. Substituindo isso na equação, o termo convectivo desaparece explicitamente do sistema: ele já está embutido na escolha de $\mathbf{x}_d$. O sistema linear passa a ter a mesma estrutura ($M/dt + K/Re$ no bloco de velocidade, acoplamento de pressão inalterado), só troca o RHS de $\frac{1}{dt}M\mathbf{v}^n - vg\mathbf{v}^n$ (advecção explícita) para $\frac{1}{dt}M\mathbf{v}_\star^n$ (velocidade interpolada no pé da característica).

2.1 O sistema em forma de blocos: antes e depois

Vale deixar isso explícito, porque é a mudança inteira em uma única equação. Hoje (`build_system_matrix + time_step` em `solver.py`), o sistema resolvido a cada passo é

$$\underbrace{\begin{bmatrix} M/dt + K/Re & 0 & -\beta G_x \\ 0 & M/dt + K/Re & -\beta G_y \\ \beta D_x & \beta D_y & 0 \end{bmatrix}}_A \begin{bmatrix} v_x^{n+1} \\ v_y^{n+1} \\ \tilde{p}^{n+1} \end{bmatrix} = \begin{bmatrix} \frac{1}{dt} M v_x^n - vg v_x^n \\ \frac{1}{dt} M v_y^n - vg v_y^n \\ 0 \end{bmatrix},$$

$$vg = \text{diag}(v_x^n) G_{vx} + \text{diag}(v_y^n) G_{vy}$$

(a advecção linearizada, montada a partir de v_x^n, v_y^n e das matrizes convectivas G_{vx}, G_{vy} de `assembly.py`, entra só no RHS – A nunca teve termo advectivo).

Com semi-Lagrangeano, **A não muda nada**: os mesmos blocos $M/dt + K/Re$ e o mesmo acoplamento de pressão. Muda só o RHS, que troca v_x^n, v_y^n (valores no próprio nó) pelos valores interpolados no pé da característica, e o termo vg some por completo – não só de A , mas do RHS também:

$$\begin{bmatrix} M/dt + K/Re & 0 & -\beta G_x \\ 0 & M/dt + K/Re & -\beta G_y \\ \beta D_x & \beta D_y & 0 \end{bmatrix} \begin{bmatrix} v_x^{n+1} \\ v_y^{n+1} \\ \tilde{p}^{n+1} \end{bmatrix} = \begin{bmatrix} \frac{1}{dt} M v_{x\star}^n \\ \frac{1}{dt} M v_{y\star}^n \\ 0 \end{bmatrix},$$

onde $v_{x\star}^n, v_{y\star}^n$ são os campos interpolados nos pés das características (Seção 3), calculados fora do sistema linear, num pré-processamento por nó.

Duas consequências diretas dessa comparação:

- `build_system_matrix` não precisaria de nenhuma mudança – é puramente a montagem de A , que já é a mesma nos dois casos;
- as matrizes G_{vx}, G_{vy} (produzidas por `assembly.assemble_mini`) deixariam de ser usadas no fluxo de `time_step` assim que a advecção explícita for removida – hoje elas só servem para montar vg . **Decisão** (§3.8): as duas abordagens ficam lado a lado durante a validação, então G_{vx}, G_{vy} continuam montadas e em uso normalmente até a migração ser confirmada – só remover depois.

3 Estratégia de implementação proposta

Seguindo a separação de responsabilidades do projeto (ver README – `mesh.py/assembly.py/boundary.py` não sabem de config nem de I/O, cada um faz uma coisa), a proposta é um módulo novo, `src/femns/semi_lagrangian.py`, com lógica pura de malha + interpolação – sem saber de `torch` esparsos, sistema global ou BiCGSTAB. `solver.py` consome sua saída.

3.1 Passo 1 – Backtrace: encontrar o ponto de partida

Para cada nó $\mathbf{x}_n$ (incluindo os nós de centroide/bolha do elemento MINI, já que vx, vy têm um valor em cada um), calcular

$$\mathbf{x}_d = \mathbf{x}_n - dt \mathbf{v}(\mathbf{x}_n, t^n).$$

Essa é a aproximação de Euler (1ª ordem). **Decisão**: usar Euler explícito nesta primeira implementação – mais simples de codificar e testar, com uma única avaliação de velocidade por nó. Uma variante mais precisa, RK2/ponto médio (estimar $\mathbf{x}_d$ com Euler, avaliar a velocidade no meio do caminho e refazer o passo), fica como evolução futura caso a validação da §3.8 mostre erro de posição relevante – não é necessária para a primeira versão funcional.

3.2 Passo 2 – Localizar o elemento que contém o ponto de partida

Esta é a parte algoritmicamente mais delicada, e a única que introduz um algoritmo de natureza diferente de tudo que já existe no projeto (que é montagem vetorizada + álgebra linear). Vale detalhar.

Por que não busca global. Testar os `ne` elementos para cada nó é $O(n_{\text{points}} \times ne)$ – na malha do Poiseuille já mencionada no README ($n \approx 19$ mil incógnitas) isso é inviável por passo de tempo. A alternativa padrão em métodos de características para FEM é a **busca por caminhada** (*walking search*): partir do elemento que já se conhece (o elemento ao qual o próprio nó $\mathbf{x}_n$ pertence) e "andar" na malha, elemento vizinho por elemento vizinho, na direção de $\mathbf{x}_d$, usando só conectividade local – custo $O(k)$ por nó, onde k é o número de elementos cruzados (tipicamente pequeno, porque o deslocamento $dt|\mathbf{v}|$ costuma ser da ordem de uma fração do tamanho de elemento h).

Coordenadas baricêntricas reaproveitando `assembly.py`. Para um triângulo de vértices i, j, k (ordenados como em `IEN`), as coordenadas baricêntricas de um ponto $\mathbf{x} = (x, y)$ são razões de área com sinal:

$$\lambda_i(\mathbf{x}) = \frac{\text{Area}(\mathbf{x}, j, k)}{\text{Area}(i, j, k)}, \quad \lambda_j(\mathbf{x}) = \frac{\text{Area}(i, \mathbf{x}, k)}{\text{Area}(i, j, k)}, \quad \lambda_k(\mathbf{x}) = \frac{\text{Area}(i, j, \mathbf{x})}{\text{Area}(i, j, k)},$$

com $\lambda_i + \lambda_j + \lambda_k = 1$ (só precisa calcular duas, a terceira sai por diferença). Usando a área com sinal $\text{Area}(a, b, c) = \frac{1}{2}[(x_b - x_a)(y_c - y_a) - (x_c - x_a)(y_b - y_a)]$, essas razões coincidem com as funções de forma P1 do elemento, $\lambda_i \equiv N_i$, que são exatamente lineares nos mesmos coeficientes b_i, c_i que `assembly.assemble_mini` já calcula para montar K e M (`bi = Y[vj]-Y[vk]`, `ci = X[vk]-X[vj]`, antes do `torch.abs` aplicado à área – aqui precisamos do sinal, não do valor absoluto). Como $\lambda_i(x_i, y_i) = 1$ e λ_i é afim com gradiente $(b_i, c_i)/(2A)$, dá para escrever, sem precisar do termo independente a_i da forma padrão $N_i = (a_i + b_i x + c_i y)/(2A)$:

$$\lambda_i(\mathbf{x}_d) = 1 + \frac{b_i(x_d - x_i) + c_i(y_d - y_i)}{2A},$$

e de forma análoga para λ_j, λ_k trocando (i, b_i, c_i) por (j, b_j, c_j) e (k, b_k, c_k) . O ponto de reaproveitar os coeficientes de `assembly.py` é só evitar duas implementações divergentes do mesmo elemento – vale bater esse cálculo contra a fórmula de área com sinal do parágrafo anterior num teste antes de decidir qual usar.

Algoritmo de caminhada. Por nó, partindo do elemento ao qual o próprio nó pertence:

1. Calcular $(\lambda_0, \lambda_1, \lambda_2)$ de $\mathbf{x}_d$ em relação aos 3 vértices do elemento atual (o nó de centroide/bolha não entra aqui, só na interpolação do Passo 3).
2. Se $\lambda_0, \lambda_1, \lambda_2 \geq -\varepsilon$ (uma tolerância numérica pequena, não ≥ 0 estrito – ver nota de robustez abaixo), o ponto está dentro: parar.
3. Caso contrário, seja $m = \arg \min_i \lambda_i$ o vértice de coordenada mais negativa. Isso indica que o ponto está do lado de fora da face oposta a m – na convenção de `mesh.montar_EToE` (`faces_locais = [(0,1), (1,2), (2,0)]`, face f liga os vértices locais f e $(f+1) \bmod 3$), a face oposta ao vértice local m é a face $(m+1) \bmod 3$. Seguir para `EToE[elem_atual, (m+1)%3]`.
4. Se `EToE` indicar -1 (face de contorno), o ponto de partida caiu fora do domínio – tratar como §3.5, não continuar a caminhada.
5. Repetir a partir do passo 1 no novo elemento, até localizar ou sair do domínio.

Notas de robustez (relevantes antes de codificar).

- **Tolerância na comparação com zero:** como as coordenadas vêm de aritmética de ponto flutuante, um ponto exatamente sobre uma aresta compartilhada (comum quando o deslocamento por passo é pequeno e $\mathbf{x}_d$ cai perto do próprio nó de origem) pode dar λ_i ligeiramente negativa por erro de arredondamento mesmo estando "dentro" – daí a tolerância ε no critério de parada, em vez de comparação estrita com zero.
- **Limite de iterações:** em malhas muito distorcidas ou perto de cantos não-convexos, a heurística "andar na direção da coordenada mais negativa" pode, em casos patológicos, não convergir monotonicamente (Devillers, Pion & Teillaud, 2001, documentam isso para busca por caminhada em triangulações e propõem variantes mais robustas, ex. *orientation walk*). Na prática, para uma malha razoável (a exemplo da gerada por Gmsh no projeto), poucos saltos bastam – mas vale por um limite de iterações (ex. proporcional a $\sqrt{ne}$) com um *fallback* explícito (busca local nos vizinhos de `NToN`, ou marcar como não localizado e cair no tratamento de fora-do-domínio) em vez de deixar o laço rodar indefinidamente.

- **EToE ainda não está conectado ao pipeline:** hoje `montar_EToE` existe e é testada (`tests/test_mesh.py`), mas `scripts/run_simulation.py` não a chama – só chama `montar_NTtoN`. Parte do trabalho de implementação é efetivamente construir EToE no setup (sobre `mesh.IEN`, antes do `elem_mini` acrescentar o nó de centroide, já que faces são definidas pelos 3 vértices), do mesmo jeito que NTtoN já é construído hoje.

3.3 Passo 3 – Interpolar a velocidade no pé da característica

Comparado ao Passo 2, este é simples: uma vez que se sabe o elemento e as coordenadas baricêntricas $(\lambda_i, \lambda_j, \lambda_k)$ de $\mathbf{x}_d$, resta avaliar o campo v_x^n, v_y^n nesse ponto.

Fórmula (P1 + bolha). O ponto importante é que v_x^n, v_y^n já são funções do espaço MINI – a mesma base usada para montar K, M, G_x, G_y em `assembly.py` –, então interpolar não é ajustar uma função nova aos valores nodais, é simplesmente **avaliar a função que já existe** nesse ponto, usando sua própria expansão de base:

$$v_\star(\mathbf{x}_d) = \lambda_i v_i + \lambda_j v_j + \lambda_k v_k + \underbrace{27 \lambda_i \lambda_j \lambda_k}_{\text{bolha}} v_b,$$

onde v_i, v_j, v_k são os valores nodais nos 3 vértices do elemento e v_b é o valor no nó de centroide desse elemento (a 4ª coluna de `IEN`, criada por `mesh.elem_mini`). A função bolha $27 \lambda_i \lambda_j \lambda_k$ é a mesma que aparece implícita nos coeficientes 9/10, 27/10, 81/10 de `assembly.assemble_mini` – vale 1 no centroide ($\lambda_i = \lambda_j = \lambda_k = 1/3$) e 0 em toda a borda do triângulo (basta um $\lambda = 0$).

Por que incluir a bolha, e o que v_b significa. Isso resolve a dúvida deixada em aberto numa versão anterior deste documento ("interpolar só a parte P1 ou incluir a bolha?"): a resposta é incluir, e não é uma escolha de projeto – é uma questão de consistência. v_b **não** é a velocidade física no ponto central do elemento; é o coeficiente modal da bolha na representação MINI do campo (por isso o método funciona mesmo v_b não tendo interpretação nodal direta). Descartar o termo da bolha na interpolação seria avaliar uma função *diferente* da que o resto do solver está de fato resolvendo – inconsistente com a própria discretização. Como $\lambda_i, \lambda_j, \lambda_k$ já foram calculados no Passo 2, o termo extra é barato (mais uma multiplicação por nó).

Higiene numérica. A tolerância ε do critério de parada do Passo 2 permite λ_i ligeiramente negativa ($-\varepsilon \leq \lambda_i < 0$) para pontos perto de uma aresta. Antes de usar os λ 's como pesos aqui, vale recortá-los para $[0, 1]$ (`torch.clamp`) e – se a soma não bater exatamente em 1 por causa do recorte – renormalizar, para garantir que a parte P1 da interpolação seja sempre uma combinação convexa dos 3 valores nodais (sem overshoot introduzido pelo próprio critério de tolerância, distinto do overshoot inerente à interpolação, comentado abaixo).

Vetorização. Ao contrário do Passo 2 (caminhada, sequencial em número de saltos), este passo é uma soma ponderada simples – assim que se tem, para cada nó, o índice do elemento encontrado e os $(\lambda_i, \lambda_j, \lambda_k)$, a interpolação de todos os nós de uma vez é um **gather** dos 4 valores nodais por elemento (3 vértices + bolha) seguido de produto e soma, no mesmo estilo vetorizado que `assembly.py` já usa para montar as matrizes de elemento. Não há laço nesta etapa.

Ordem de precisão e overshoot. Interpolação P1 (+ bolha) é exata para campos que já são combinação linear dessa base, mas introduz erro de truncamento de $O(h^2)$ para campos mais suaves – esse é o mecanismo concreto da "difusão numérica" mencionada na §4 (Impacto esperado). Um efeito relacionado, comum em métodos semi-Lagrangeanos, é *overshoot*: interpolação linear não garante que $v_\star$ fique entre o mínimo e o máximo dos valores nodais do elemento em todas as situações combinadas com o termo de bolha (que não é monotônico). Staniforth & Côté

(1991), a referência clássica de revisão de esquemas semi-Lagrangianos, discutem esse problema e limitadores para mitigá-lo – não é necessário para uma primeira versão funcional, mas vale ter em mente se aparecerem picos não-físicos de velocidade perto de gradientes acentuados (ex. camada limite).

Teste. Seguindo o padrão de `tests/`: escolher valores nodais arbitrários v_i, v_j, v_k, v_b para um triângulo de referência (os mesmos usados em `tests/test_assembly.py`), calcular v_* à mão para um ponto interno conhecido (ex. o próprio centroide, onde a fórmula colapsa para $\frac{1}{3}(v_i + v_j + v_k) + v_b$) e comparar com a saída da função de interpolação – valida a fórmula isolada do Passo 2 (localização).

3.4 Passo 4 – Trocar o termo convectivo em `solver.py`

Em `time_step`, substituir:

```
vg = torch.mm(vx_diag, Gvx) + torch.mm(vy_diag, Gvy)
b_sup = (1/dt) * torch.mm(M, vx.unsqueeze(1)) - torch.mm(vg, vx.unsqueeze(1))
```

por algo como:

```
vx_star, vy_star = semi_lagrangian.backtrace_and_interpolate(...)
b_sup = (1/dt) * torch.mm(M, vx_star.unsqueeze(1))
```

A matriz A (`build_system_matrix`) não muda – ela já não tinha advecção. O ganho de Δt maior sem instabilidade vem inteiramente dessa troca no RHS.

3.5 Pontos de partida fora do domínio

Perto de contornos de entrada (*inlet*), o pé da característica pode cair fora da malha – fisicamente correto (a partícula "veio de fora", carregando o valor de contorno). **Decisão:** se a caminhada sair do domínio, usar o valor de Dirichlet do contorno por onde saiu (ex. `inlet: {vx: 1.0, vy: 0.0}` no config); se o contorno não definir esse componente ali (ex. `outlet`, que só define p no exemplo do README), usar o valor atual do próprio nó de chegada como aproximação – equivalente a um passo de Euler explícito só nesse nó, ou seja, cai de volta no comportamento de hoje exatamente nos casos em que não há dado de contorno melhor para usar. `boundary.py` já resolve qual nome de contorno cada nó tem (`assign_boundary_names`) – a mesma lógica de prioridade dá para reaproveitar aqui.

3.6 Vetorização em GPU / `torch`

A caminhada (Passo 2) é sequencial por natureza (número de passos varia por nó), o que não vetoriza tão bem quanto o resto do projeto (que é 100% montagem vetorizada em `torch`, ver `assembly.py`). Estratégia realista: vetorizar *entre* nós (todos os nós fazem "um passo de caminhada" em paralelo, um tensor booleano marca quais já terminaram) e iterar até que todos converjam – número de iterações do laço = máximo de saltos de elemento entre todos os nós, não `npoints × ne`. Vale prototipar em CPU/`numpy` primeiro (mais fácil de depurar com `prints/asserts`) e só depois portar para `torch` caso o custo por passo de tempo se mostre relevante.

3.7 Testes

Seguindo o padrão de `tests/` (que testa `assembly.py` contra valores analíticos de um triângulo de referência, sem depender de malha `.msh`): um teste unitário com 1–2 triângulos e um campo de velocidade constante (translação pura) tem solução fechada para o ponto de partida e para a interpolação – bom primeiro caso para validar localização + interpolação isoladamente, antes de plugar em `time_step`.

3.8 Estratégia de transição e validação

Decisão: as duas abordagens de advecção ficam lado a lado durante a validação, em vez de substituir a atual de uma vez. Um parâmetro novo no config (ex. `simulation.advection: explicit | semi_lagrangian`, junto de `dt/iterations/reynolds`) seleciona qual caminho `time_step` usa a cada passo – `explicit` continua montando vg com G_{vx}, G_{vy} como hoje, `semi_lagrangian` usa `semi_lagrangian.backtrace_and_interpolate`. Isso mantém as matrizes G_{vx}, G_{vy} em uso normalmente até a migração ser confirmada.

Validação proposta, no mesmo espírito da tabela comparativa que valida o BiCGSTAB em `docs/bicgstab_solver.pdf`: rodar o Poiseuille de exemplo (`configs/poiseuille.yaml`, com `iterations` reduzido para testar rápido) com `dt` pequeno – onde a advecção explícita atual é estável e portanto serve de referência – nas duas abordagens, e comparar posição de nó (efeito do `moving_point`) e campos de velocidade/pressão ao longo das iterações. Só depois de bater (dentro de uma tolerância compatível com a ordem de erro esperada da interpolação P1+bolha, ver §3.3) faz sentido testar `dt` maior – onde as duas deixam de ser diretamente comparáveis, já que aí é onde o semi-Lagrangeano deveria continuar estável e o explícito não. Remover $G_{vx}, G_{vy}/vg$ e a opção `explicit` do config só depois dessa validação, não antes.

4 Impacto esperado e trade-offs

- **A favor:** remove a restrição de CFL da advecção, permitindo `dt` maior (potencialmente relevante já que o README aponta `dt` pequeno como parte do custo por iteração).
- **Contra:** interpolação (mesmo linear) introduz difusão numérica – a solução deixa de ser idêntica à advecção Galerkin explícita atual, mesmo em `dt` pequeno onde ambas seriam estáveis. Isso é uma mudança de *numérica*, não só de performance.
- Complexidade nova: busca por elemento (Passo 2) é um algoritmo diferente de tudo que existe hoje no projeto (que é só montagem vetorizada + solve linear) – maior superfície para bugs sutis (ex.: caminhada que não converge em malhas muito distorcidas).

5 Referências

1. Pironneau, O. (1982). *On the transport-diffusion algorithm and its applications to the Navier–Stokes equations*. Numerische Mathematik, 38(3), 309–332. – artigo original que introduz o método de características (semi-Lagrangeano) aplicado a elementos finitos para Navier–Stokes; base teórica direta para a formulação da §2.
2. Donea, J., & Huerta, A. (2003). *Finite Element Methods for Flow Problems*. Wiley. – capítulo sobre métodos Characteristic-Galerkin/semi-Lagrangeanos, com algoritmos práticos de localização de ponto em malhas de elementos finitos (base do Passo 2) e discussão de erro de interpolação/difusão numérica (§4).
3. Devillers, O., Pion, S., & Teillaud, M. (2001). *Walking in a triangulation*. International Journal of Foundations of Computer Science, 13(2), 181–199. – análise da busca por caminhada (*walking search*) usada no Passo 2: condições de convergência, casos patológicos e variantes mais robustas que a heurística "coordenada mais negativa" usada aqui.
4. Staniforth, A., & Côté, J. (1991). *Semi-Lagrangian integration schemes for atmospheric models – a review*. Monthly Weather Review, 119(9), 2206–2223. – revisão clássica de esquemas semi-Lagrangeanos; discute ordem de precisão da interpolação no pé da característica e o problema de overshoot citado no Passo 3.